

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Prof. Dr. Ulrich Schäfer

Projektarbeit Mobile & Ubiquitous Computing SoSe 2026

Bachelorstudiengänge Medieninformatik, Industrie-4.0-Informatik,
Künstliche Intelligenz, Ingenieurpädagogik

21. Mai – 30. Juni 2026



Abbildung 1: Waveshare ESP32-S3 1.69inch Touch Display Development Board

1 Aufgabenstellung

Ziel der Projektarbeit in Mobile & Ubiquitous Computing ist es, für die Waveshare ESP32-Hardware ("ESP32-S3 1.69inch Touch Display Development Board") je Team zwei Spiele mit Lagesensor(IMU)-Steuerung und mit Spielstands-Anzeige und Kontrolle per MQTT zu entwickeln.

Die Spielentwicklung erfolgt mit der Arduino-Umgebung (C++) und GFX-Bibliothek.

- Spiel 1: zufällig in Labyrinth verteilte Kekse sammeln
- Spiel 2: zufällig auf der Bildschirmfläche verteilte Kekse mit einer "Kugel" aufsammeln

In beiden Spielvarianten soll eine Kugel per Beschleunigungssensor in einer Art Physik-Simulation (je stärker Neigung, desto stärker die Beschleunigung, bei Spiel 2 auch mit Reflexion an Bildschirmrand) bewegt werden.

Um auch den IoT-Aspekt der Lehrveranstaltung abzudecken, soll die Bewegungsinformationen, Zeit und Spielstand (Punkte, Runden) per MQTT an einen "digitales Dashboard" auf einem Android-Gerät sowie auf einem Node-RED-Dashboard auf dem PC/Laptop ebenfalls per MQTT angezeigt werden. In der Gegenrichtung soll über das jeweilige Dashboard auch das Spiel ausgewählt, gestartet und der Spielernamen sowie die Anzahl der Kekse und beim Labyrinth Wandstärke eingegeben werden können.

2 Weitere Informationen und Implementierungshinweise

Das ESP32-S3 1.69inch Touch Display Development Board hat ein LCD-Display mit einer Bildschirmauflösung von 240×280 Pixel (ST7789V2) sowie einen Beschleunigungssensor (QMI8658 6-axis IMU: 3-axis gyroscope + 3-axis accelerometer).

Info-Seite des Herstellers mit Download der Bibliotheken und Beispiele:

<https://docs.waveshare.com/ESP32-S3-Touch-LCD-1.69> [4].

▼ Bitte unbedingt die vier Bibliotheken aus <https://github.com/waveshareteam/ESP32-S3-Touch-LCD-1.69> unter Arduino/libraries (SensorLib, Mylibrary, lvgl und GFX_Library_for_Arduino) verwenden und NICHT durch die Arduino-IDE updaten lassen (einfach wieder aus dem git draufkopieren, falls versehentlich update erfolgt ist; lvgl wird nur zum Ausprobieren einiger Beispiele gebraucht).

Hinweis zur Entwicklungsumgebung: Bitte verwenden Sie die Arduino IDE (Version nicht zu alt, z.B. 2.3.7 oder 2.3.8), Doku: [1].

▼ Im Arduino Boards Manager bitte "esp32 by Espressif Systems" in der Version 3.3.2 verwenden. ACHTUNG: mit der neuen Version 3.3.8 funktionieren die auf der Waveshare downloadbaren 11 Beispielskette NICHT, mit Version 3.3.2 funktionieren alle 11 Beispiele von Waveshare (Stand 17.05.2026).

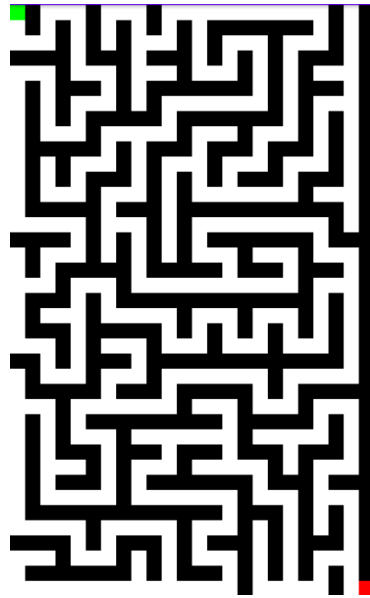


Abbildung 2: Zufällig erzeugtes Labyrinth

Board-Bezeichnung in Arduino: "Waveshare ESP32-S3-Touch-LCD-1.69".

Wer im Zweierteam gerade keinen Zugriff auf die Hardware hat, kann z.B. in wokwi (kostenlosen account anlegen) mit der GFX-Bibliothek entwickeln und testen. Unter <https://wokwi.com/esp32> finden Sie Beispielprojekte mit der Adafruit GFX library als Basis. Falls Sie in GFX Text ausgeben wollen, finden Sie Links zu weiteren Fonts in der Datei README.md in GFX_Library_for_Arduino.

Inspiration zur Physiksimulation per Accelerometer finden Sie z.B. als Java-Code von Google: <https://github.com/googlearchive/android-AccelerometerPlay>.

Das Labyrinth im Spiel soll in jedem Durchgang zufällig **neu** erzeugt werden. Sie dürfen fürs Zeichnen von Labyrinth bzw. Kugel-Physiksimulation sich helfen lassen von LLMs (z.B. die freie Version von ChatGPT), also z.B. mit Tiefensuche ein zufälliges Labyrinth berechnen; die verwendeten Prompts sind im Anhang zur Ausarbeitung (zählt nicht mit zur Seitenberechnung) anzugeben. Die schriftliche Ausarbeitung fertigen Sie jedoch bitte ohne „KI“ selbständig an, Sie sollen dort insbesondere auch die Physiksimulation erklären.

Bitte definieren Sie Parameter wie Bildschirmauflösung in globalen Konstanten am Codeanfang, nicht fixe numerische Werte im Programmcode!

Für WLAN- und MQTT-Broker-Credentials gibt es eine eigene Datei (s.u.).

MQTT, HiveMQ und Dashboards

Sie sollen zwei von mehreren möglichen MQTT-basierten Dashboard-Varianten entwickeln. Als Broker soll HiveMQ (freie Cloud-Version; <https://www.hivemq.com/products/mqtt-cloud-broker/>) zum Einsatz kommen. Sie können zum lokalen Testen natürlich auch Mosquitto (<https://mosquitto.org/>) oder andere installieren.

Hinweise zu HiveMQ: Für den Broker-Zugriff aus der MQTT-Bibliothek (z.B. PubSub-Client für den ESP32/Arduino) benötigen Sie das Letsencrypt-Zertifikat sowie einen eigenen HiveMQ-Cloud-Zugang (kostenlos; Registrierung erforderlich), es liegt bereits in der Datei `esp32/wifi_mqtt_secrets.h` vor, die Sie bitten in Ihrem Sketch includen (eine Verzeichnisebene tiefer schieben).

Jedes Team muss ein Node-RED-Dashboard entwickeln, das andere ist frei wählbar. Die Dashboards soll beide Spiele steuern und deren Spielstände, Benutznamen usw. anzeigen: Node-RED Dashboard ("low code"; [3]), dazu entweder Android-App selbst entwickelt oder MQTT Dash¹ ("no code") oder Web-basiert (dafür muss ggf. MQTT over Websocket im Broker aktiviert werden, meist anderer Port). Falls Sie eine Android-Lösung selbst entwickeln möchten (Java/Kotlin), wird die Nutzung der HiveMQ-Bibliothek empfohlen, die Eclipse-paho-Variante ist etwas hakelig, funktioniert mit der passenden Version aber auch.

Falls im Labor mit dem IoT-WLAN Probleme auftauchen, wäre hier auf Anfrage Fall-back zum lokalen WLAN `ki-lokal` im DC1.07 ohne Internet-Zugriff, Broker dann auf `192.168.1.10`, möglich.

Hinweis zu Node-RED: Deployen nach Änderung des Flows nicht vergessen... Speichern des definierten Node-RED-Flows für die Abgabe über das Export-Menü (JSON-Format).

Handling der Hardware

⚠ Die Unterseite der Boards ist nicht isoliert, bitte achten Sie darauf, dass keine metallischen Gegenstände die Unterseite berühren und so schlimmstenfalls ein Kurzschluss entsteht.

Möglichst Stecker auf der PC/-Laptop-Seite ziehen (USB-A), Display-seitig (USB-C) möglichst stecken lassen (außer für Transport).

Display-Schutzfolien bitte nicht entfernen, Touch wird nicht benötigt, Bedienung erfolgt per Lagesensor bzw. MQTT und das Dashboard.

3 Projektplanung, Vorbereitung und Vorgehen

Die Gruppeneinteilung erfolgt in moodle (Zweierteams).

Bitte erstellen Sie je Team zunächst eine Projektplanung mit Aufgabenverteilung und dokumentieren Sie diese. Wie Sie die Arbeit aufteilen, ist Ihnen überlassen, aber bitte gehen Sie in der Dokumentation darauf ein, wer welche Projektteile implementiert und dokumentiert hat (Zuständigkeiten).

Auch bezüglich der Dokumentation selbst sowie des Codes muss hervorgehen, wer für was verantwortlich ist/war. Generell sollen die beiden Spiele zunächst getrennt

¹<https://play.google.com/store/apps/details?id=net.routix.mqttdash&hl=de>

voneinander entwickelt werden, aber in letztendlich in einen Sketch zusammengeführt werden. Ein Arduino-Sketch kann aus mehreren cpp- bzw. Header-Dateien bestehen. Darüber hinaus soll im Team Abstimmung z.B. über die MQTT-Topics und die NodeRED- bzw. Android-Dashboards erfolgen. Die beiden letzteren sollen für beide Spiele (inkl. Auswahl des Spiels) funktionieren.

Falls Sie ihren eigenen Rechner verwenden möchten, installieren Sie vor Beginn Ihrer Recherchephase Zotero [5] oder alternativ JabRef [2], um Ihre Quellen (auch Web-Quellen wie Forenbeiträge aus StackOverflow oder ähnlichen) zu verwalten und einfach in Ihre Projektdokumentation mit aufzunehmen.

4 Dokumentation und Abgabe

Schriftliche Ausarbeitung ca. 10 Seiten inkl. ca. 3–4 Screenshot(s), bitte zusätzlich auch den Node-RED-Flow grafisch darstellen (ebenfalls Screenshot), ESP32-Code in den Anhang (zählt nicht zu den ca. 10 Seiten), aber allen Quellcode auch in der zip-Datei abgeben.

Projektplanung, Vorgehen, Teamaufteilung beschreiben.

Codedokumentation inline im Code.

Quellen angeben im Literaturverzeichnis (auch Web-Links, in .bib-Datei aus Zotero).

Diskutieren Sie in der Dokumentation bitte ggf. auch Probleme und daraus resultierende Entscheidungen.

5 Abgabe-Checkliste

Abgabe in einer zip-Datei, PDF-Doku darin auf der obersten Verzeichnisebene.

Dateiname der hochzuladenden Datei: Gruppe_d.zip (dies ist ein regulärer Ausdruck, also konkret z.B. Gruppe_03.zip) -- Inhalt:

Quellcode, sonstige Hilfsdateien; vollständig und kommentiert, sowie die Projektdokumentation. Verzeichnisstruktur:

```
Gruppe_03.pdf # Ihre Projektdokumentation aus tex/ (PDF, KEIN WORD!!!) mit Angabe
               aller verwendeten Quellen (Gruppennummer anpassen!)
android/      # ggf. Android-App-Projektverzeichnis (Android Studio-Projekt),
               OHNE den Inhalt des Ordners app/build
esp32/        # Quellcode für ESP32-S3 (Arduino Sketch C++)
nodered/      # NodeRED Flow JSON-Datei (exportiert aus NodeRED)
prompts/      # für die von Ihnen ggf. verwendeten LLM-Transkripte
screenshots/  # Screenshots (Spiele, Dashboards, Flows, ggf. auch von MQTT Dash)
tex/          # LaTeX-Quellcode mit .bib-Datei, Screenshots/Bitmap-Dateien usw.
               # enthält bereits Vorlagen für Projektdokumentation
               projektarbeit.tex -- LaTeX-Datei als Vorlage, Beispiel, Hinweise
               quellen.bib       -- Literaturquellen (BibTeX-Format)
```

ACHTUNG: Für die Abgabe WLAN-SSID und -Passwort auf IoT-WLAN zurücksetzen (ihr privater WLAN-Zugang ist für mich nicht von Interesse, ebenso wenig Ihre HiveMQ-Cloudzugangsdaten). Dazu stehen diese initial separat in der Konfigurationsdatei `esp32/wifi_mqtt_secrets.h` bzw. bei der Abgabe im Sketch eine Verzeichnisebene tiefer.

6 Bewertungsgrundlagen

- Projektarbeits-Beschreibung, Quellenangaben, Aufbau
- Usability
- Code: Korrektheit, Vollständigkeit, Eleganz, Lösungswege
- Code-Dokumentation: inline (C++-Code)

Präsentation der Projekte und Demonstration der Anwendung durch das jeweilige Team am 1./2. Juli 2026.

Nach der Abgabe bitte regelmäßig email checken, falls Rückfragen nötig sind.

Viel Spaß & viel Erfolg!

Literatur

- [1] ARDUINO: *Arduino Language Reference*. <https://docs.arduino.cc/language-reference/>. Version: 2026, Abruf: 17.05.2026
- [2] JABREF.ORG: *JabRef – Stay on top of your literature*. <https://www.jabref.org>, Abruf: 17.05.2026. – JabRef is an open-source, cross-platform citation and reference management tool
- [3] OPENJS FOUNDATION: *Node-RED – Low-code programming for event-driven applications*. <https://nodered.org>, Abruf: 17.05.2026
- [4] WAVESHARE INTERNATIONAL LIMITED: *ESP32-S3 1.69inch Touch Display Development Board*. <https://docs.waveshare.com/ESP32-S3-Touch-LCD-1.69>. Version: 2025, Abruf: 17.05.2026
- [5] ZOTERO.ORG: *Zotero – Your personal research assistant*. <https://zotero.org>, Abruf: 17.05.2026. – Zotero is a free, easy-to-use tool to help you collect, organize, annotate, cite, and share research